レガシーコード改善はじめました

2012/4/26
アジャイルサムライ横浜道場 特別編

せとあずさ♂

今日の出来事

大中 浩行

5時間前 ・

Jenkinsの「SCMをポーリング」と「定期的に実行」を間違えて1分おきにビルドが走ってた＞＜

いいね！ · コメントする · 投稿のフォローをやめる

橋立 友宏さん、かぬか ひでとしさん、他2人が「いいね！」と言っています。

気をつけましょう

- せとあずさ♂
- Growing Object-Oriented Software,Guieded by Test (goos_jp)読書会主宰
- ぺけま http://devtesting.jp/pekema/
- yokohama.rb / shinjuku.rb
- 「みなとRuby会議」 2012/6/2
- 仕事は生産性向上とか、品質改善とか

レガシーコード

Working Effectively with Legacy Code

レガシーコード改善ガイド

保守開発のためのリファクタリング

マイケル・C・フェザーズ 著
ウルシステムズ株式会社 監訳
平澤章／越智典子／稲葉信之／田村友彦／小堀真義 訳

テストがないコードは
レガシーコードだ！

ソースコードがきれいで、良い構造であれば十分か？
——そうではない
もし、テストコードなしで大幅な修正を加えるとしたら
信じられないほどのスキルと明確な理解が必要になる

あなたも、
JavaやNETで
レガシーコードを
書いていませんか？

SE

テストがないコード

はレガシーコード

だ！

わかりました

テスト書きましょう

というわけで今日は

- レガシーコードな社内フレームワークに、TDDでテスト書きながら機能拡張したお話し

いきさつ

- よくある社内フレームワーク
  - Java
  - FilterベースのMVC + 簡易O/Rマッピング
  - 大幅に機能拡張をすることに

きっかけ

- 運用開始後にフレームワークの致命的なバグが発覚することが多く、現場からクレーム
- 自分としてもテストをがっちり書いて開発したかった

テストの方針

- 奇をてらったことはしていない
- JUnit4(hamcrest) + mockito
- ビルドはAnt
- Cobetureでカバレッジ計測
- JenkinsでCI
- インスペクションはFindBugs+CheckStyle+Jtest

- なんだかんだでAPIの仕様が設計段階でほぼfixしていたのは大きかった。
- 仕様が安定していたことにより、精度の高いテストケースを組むことが出来た

もうひとつの特徴

環境の組み合わせが

多い

- Java
  - Java5 / Java6 / Java7
- RDBMS
  - Oracle 10g / 11g
  - SQLServer 2005 / 2008 / (2012)
  - PostgreSQL 7.4 / 8.0 / 8.1 / 8.2 / 8.3 / 8.4 / 9.0 / 9.1 / (9.2)
- Servet 2.3 / 2.4 / 3

- これ、全部テストするんですか？

- してますが、何か？

java6
java7
javadoc
mssqlserver2005
mssqlserver2005.java6
mssqlserver2005.java7
mssqlserver2008
mssqlserver2008.java6
mssqlserver2008.java7
oracle10g
oracle10g.java6
oracle10g.java7
oracle11g
oracle11g.java6
oracle11g.java7
postgresql74
postgresql74.java6
postgresql74.java7
postgresql80
postgresql80.java6
postgresql80.java7
postgresql81
postgresql81.java6
postgresql81.java7
postgresql82
postgresql82.java6
postgresql82.java7
postgresql83
postgresql83.java6
postgresql83.java7
postgresql84
postgresql84-java6
postgresql84-java7
postgresql91
postgresql91-java6
postgresql91-java7

- Eclipse上ではPostgereSQL9.0+Java5で開発し、残りの組み合わせはCI環境で実施
- Jenkinsでビルドする際に、Ant にシステムプロパティを渡してデータソース、クラスパス等を切り替え
- パラメータの設定は多分もっと楽な方法はあるが...

- SVNにコミットすると、まず Java5+PostgreSQL9.0でビルドを行う
  - この時点ではRDBMS固有のロジックはモック(Test Double)を使用
- その後、JVM別、RDBMS別のテストを8スレッド並行で行う。
- 並行実行のためにRDBMSのスキーマは別々
- 総実行テストケース数 6,900 件以上！

とはいっても

最初からこうだった

わけじゃない

- (PostgreSQLだけおさえといて、後は結合でいいだろう....)
- 「OracleとSQLServerも見といてくださいね」
- 「えっ」
- 「Java6だけじゃなくてJava5もサポートしてくださいね」
- 「えっ」

- 動作環境の組み合わせが増えると、色々ある

- Java6とJava5でHashMapの並び順が逆だったりとか...
- MSSQLServerのDATETIMEの精度が3.33ミリ秒でうまく楽観排他できなかったりとか...
- Java7とSQLServer2008の組み合わせでJDBCドライバのバグ踏んだりとか...

- 多様な環境をサポートするのはやっぱ大変

- でもですね、

- ユニットテストなしに、動作環境の多様な組み合わせでの動作保証は出来なかった
- その後のバグ修正、機能拡張が非常にスムーズになった
- というのは、自信をもって言える

- 先日SQLServerのJDBCドライバに大バグがあることが発覚して、バージョンアップしたドライバでの動作検証が必要だったのですが、15分で検証終了しました。

- 現時点での課題は、データベースの追加が面倒くさいこと

- DDLの投入は自動化してるが、スキーマ追加が手作業になっている
- テスト用のデータソースの設定が、フレームワークのいけてない仕様をひきづって、面倒くさくなっている
- 鋭意改善中

- ユニットテストが全パスしたら、Seleniumでのテスト用のアプリケーションをデプロイし、そのアプリケーションに対してのテストを行う
- ...というのを、今やっているところ

乞うご期待